Homework 1: 混凝土抗压强度预测

此文档展现作业中的完整流程运行步骤，具体展现为：

1. 读取 UCI Concrete Compressive Strength 数据集
2. 按题目要求随机shuffle，以 80% 作为训练集、后 20% 作为测试集
3. 在训练集上做特征相关性分析并可视化
4. 对输入特征做标准化，去空值异常值，对目标值做 `log1p + 标准化` 处理
5. 训练 PyTorch MLP、Linear回归、多种设置的MLP，并使用验证集早停，对比多种设置 MLP 和 Linear 层
6. 在测试集上评估并绘制训练曲线、预测散点图
7. 导出 notebook 运行产生的结果文件
8. **对比实验——使用与结果相关性最大的几个特征预测，测试与评估**

导入必要的库

In [38]:
```
from pathlib import Path
import json
import random

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

plt.style.use("seaborn-v0_8")
torch.set_printoptions(sci_mode=False)
```

读取数据以及设置超参数

In [39]:
```
DATASET_PATH = Path("dataset/Concrete_Data.xls")
OUTPUT_DIR = Path("notebook_outputs")
SEED = 42

EPOCHS = 1200
BATCH_SIZE = 64
LR = 5e-4
WEIGHT_DECAY = 1e-3
PATIENCE = 120
VAL_RATIO = 0.15

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

撰写必要的训练、测试和可视化函数

In [40]:
```
def set_seed(seed: int = SEED) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```python
class StandardScaler:
    def __init__(self) -> None:
        self.mean = None
        self.std = None

    def fit(self, array: np.ndarray) -> None:
        self.mean = array.mean(axis=0, keepdims=True)
        self.std = array.std(axis=0, keepdims=True)
        self.std[self.std == 0] = 1.0

    def transform(self, array: np.ndarray) -> np.ndarray:
        return (array - self.mean) / self.std

    def fit_transform(self, array: np.ndarray) -> np.ndarray:
        self.fit(array)
        return self.transform(array)

    def inverse_transform(self, array: np.ndarray) -> np.ndarray:
        return array * self.std + self.mean

class MLPRegressor2(torch.nn.Module):
    def __init__(self, input_dim: int) -> None:
        super().__init__()
        self.network = torch.nn.Sequential(
            torch.nn.Linear(input_dim, 64),
            torch.nn.ReLU(),
            torch.nn.Linear(64, 32),
            torch.nn.ReLU(),
            torch.nn.Linear(32, 1),
        )

    def forward(self, inputs: torch.Tensor) -> torch.Tensor:
        return self.network(inputs)

class MLPRegressor1(torch.nn.Module):
    def __init__(self, input_dim: int) -> None:
        super().__init__()
        self.network = torch.nn.Sequential(
            torch.nn.Linear(input_dim, 64),
            torch.nn.GELU(),
            torch.nn.Linear(64, 32),
            torch.nn.GELU(),
            torch.nn.Linear(32, 16),
            torch.nn.GELU(),
            torch.nn.Linear(16, 1),
        )

    def forward(self, inputs: torch.Tensor) -> torch.Tensor:
        return self.network(inputs)


class MLPRegressor(torch.nn.Module):
    def __init__(self, input_dim: int) -> None:
        super().__init__()
        self.network = torch.nn.Sequential(
            torch.nn.Linear(input_dim, 64),
            torch.nn.ReLU(),
            torch.nn.Linear(64, 32),
            torch.nn.ReLU(),
```

```
            torch.nn.Linear(32, 16),
            torch.nn.ReLU(),
            torch.nn.Linear(16, 1),
        )

    def forward(self, inputs: torch.Tensor) -> torch.Tensor:
        return self.network(inputs)


def save_correlation_artifacts(train_df: pd.DataFrame, target_col: str, output_d
    correlations = (
        train_df.corr(numeric_only=True)[target_col]
        .drop(target_col)
        .sort_values(key=lambda series: series.abs(), ascending=False)
    )
    correlation_df = correlations.rename("correlation_with_target").reset_index(
    correlation_df.columns = ["feature", "correlation_with_target"]
    correlation_df.to_csv(output_dir / "correlation_rank.csv", index=False, enco

    plt.figure(figsize=(10, 5))
    plt.barh(correlation_df["feature"], correlation_df["correlation_with_target"
    plt.xlabel("Pearson correlation with target")
    plt.title("Feature Correlation Analysis")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.savefig(output_dir / "correlation_bar.png", dpi=200)
    plt.show()
    plt.close()
    return correlations


def plot_loss_curve(history: dict[str, list[float]], output_dir: Path) -> None:
    plt.figure(figsize=(8, 5))
    plt.plot(history["train_loss"], label="Train Loss", linewidth=2)
    plt.plot(history["val_loss"], label="Validation Loss", linewidth=2)
    plt.xlabel("Epoch")
    plt.ylabel("MSE Loss")
    plt.title("Training Curve")
    plt.legend()
    plt.tight_layout()
    plt.savefig(output_dir / "loss_curve.png", dpi=200)
    plt.show()
    plt.close()


def plot_predictions(y_true: np.ndarray, y_pred: np.ndarray, output_dir: Path) -
    min_value = min(float(y_true.min()), float(y_pred.min()))
    max_value = max(float(y_true.max()), float(y_pred.max()))

    plt.figure(figsize=(6, 6))
    plt.scatter(y_true, y_pred, alpha=0.75, color="#d67229", edgecolors="none")
    plt.plot([min_value, max_value], [min_value, max_value], linestyle="--", col
    plt.xlabel("True Strength (MPa)")
    plt.ylabel("Predicted Strength (MPa)")
    plt.title("Prediction vs Ground Truth")
    plt.tight_layout()
    plt.savefig(output_dir / "prediction_scatter.png", dpi=200)
    plt.show()
    plt.close()
```

```python
def train_model(
    model: torch.nn.Module,
    train_features: torch.Tensor,
    train_targets: torch.Tensor,
    val_features: torch.Tensor,
    val_targets: torch.Tensor,
    epochs: int = EPOCHS,
    batch_size: int = BATCH_SIZE,
    lr: float = LR,
    weight_decay: float = WEIGHT_DECAY,
    patience: int = PATIENCE,
) -> tuple[torch.nn.Module, dict[str, list[float]], int]:
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight
    loss_fn = torch.nn.MSELoss()
    history = {"train_loss": [], "val_loss": []}

    best_val_loss = float("inf")
    best_state = None
    best_epoch = 0
    remaining_patience = patience
    train_indices = np.arange(len(train_features))

    for epoch in range(1, epochs + 1):
        model.train()
        shuffled_indices = np.random.permutation(train_indices)
        epoch_losses = []

        for start in range(0, len(shuffled_indices), batch_size):
            batch_indices = shuffled_indices[start : start + batch_size]
            batch_x = train_features[batch_indices]
            batch_y = train_targets[batch_indices]

            optimizer.zero_grad()
            predictions = model(batch_x)
            loss = loss_fn(predictions, batch_y)
            loss.backward()
            optimizer.step()
            epoch_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            val_loss = loss_fn(model(val_features), val_targets).item()

        history["train_loss"].append(float(np.mean(epoch_losses)))
        history["val_loss"].append(val_loss)

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_state = {key: value.detach().cpu().clone() for key, value in mo
            best_epoch = epoch
            remaining_patience = patience
        else:
            remaining_patience -= 1
            if remaining_patience == 0:
                break

    model.load_state_dict(best_state)
    return model, history, best_epoch
```

1. 读取数据并查看基本信息

In [41]:
```
set_seed(SEED)

if not DATASET_PATH.exists():
    raise FileNotFoundError(f"Dataset not found: {DATASET_PATH}")

df = pd.read_excel(DATASET_PATH)
feature_cols = list(df.columns[:-1])
target_col = df.columns[-1]

print(f"Dataset shape: {df.shape}")
print(f"Feature count: {len(feature_cols)}")
print(f"Target column: {target_col}")
df.head()
```

```
Dataset shape: (1030, 9)
Feature count: 8
Target column: Concrete compressive strength(MPa, megapascals)
```

Out[41]:

	Cement (component 1)(kg in a m^3 mixture)	Blast Furnace Slag (component 2)(kg in a m^3 mixture)	Fly Ash (component 3)(kg in a m^3 mixture)	Water (component 4)(kg in a m^3 mixture)	Superplasticizer (component 5) (kg in a m^3 mixture)	Coa Aggreg (compon 6)(kg i m mixtu
0	540.0	0.0	0.0	162.0	2.5	104
1	540.0	0.0	0.0	162.0	2.5	105
2	332.5	142.5	0.0	228.0	0.0	93
3	332.5	142.5	0.0	228.0	0.0	93
4	198.6	132.4	0.0	192.0	0.0	97

2. 训练集 / 测试集划分

Shuffle数据集（避免数据集顺序影响），直接以 80% 作为训练集， 20% 作为测试集。

In [42]:
```
train_size = int(len(df) * 0.8)
# shuffle 数据集
df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)
train_df = df.iloc[:train_size].copy()
test_df = df.iloc[train_size:].copy()

print(f"Train size: {len(train_df)}")
print(f"Test size: {len(test_df)}")
train_df.head()
```

```
Train size: 824
Test size: 206
```

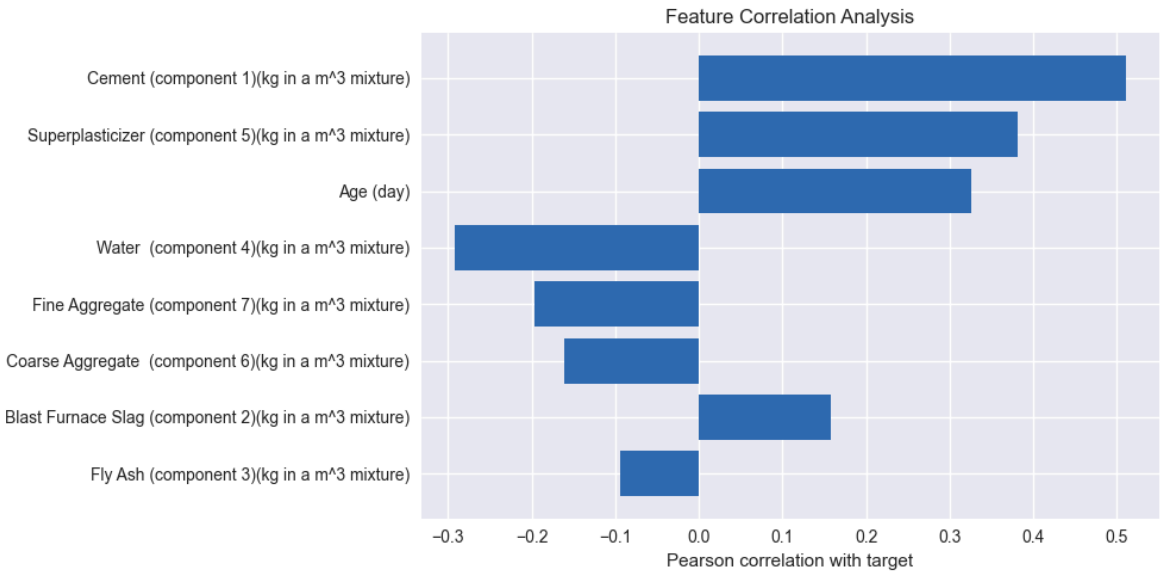
Out[42]:

	Cement (component 1)(kg in a m^3 mixture)	Blast Furnace Slag (component 2)(kg in a m^3 mixture)	Fly Ash (component 3)(kg in a m^3 mixture)	Water (component 4)(kg in a m^3 mixture)	Superplasticizer (component 5) (kg in a m^3 mixture)	Coa Aggreg (compon 6)(kg i m mixtu
0	266.0	114.0	0.0	228.0	0.0	93
1	362.6	189.0	0.0	164.9	11.6	94
2	389.9	189.0	0.0	145.9	22.0	94
3	362.6	189.0	0.0	164.9	11.6	94
4	145.0	0.0	179.0	202.0	8.0	82

3. 训练集特征相关性分析

In [43]:

```
correlations = save_correlation_artifacts(train_df, target_col, OUTPUT_DIR)
correlation_df = correlations.rename("correlation_with_target").reset_index()
correlation_df.columns = ["feature", "correlation_with_target"]
correlation_df
```



Out[43]:

	feature	correlation_with_target
0	Cement (component 1)(kg in a m^3 mixture)	0.510422
1	Superplasticizer (component 5)(kg in a m^3 mix...	0.381543
2	Age (day)	0.325834
3	Water (component 4)(kg in a m^3 mixture)	-0.291914
4	Fine Aggregate (component 7)(kg in a m^3 mixture)	-0.195972
5	Coarse Aggregate (component 6)(kg in a m^3 mi...	-0.161508
6	Blast Furnace Slag (component 2)(kg in a m^3 m...	0.157240
7	Fly Ash (component 3)(kg in a m^3 mixture)	-0.094605

4. 数据预处理

- 先检查训练集和测试集中的缺失值
- 特征缺失值使用训练集各列中位数填充，避免数据泄漏
- 目标值若缺失，则删除对应样本
- 输入特征使用训练集统计量做标准化
- 目标值先做 `log1p`，再标准化，以提高训练稳定性

In [44]:
```
train_feature_missing_before = int(train_df[feature_cols].isna().sum().sum())
test_feature_missing_before = int(test_df[feature_cols].isna().sum().sum())
train_target_missing_before = int(train_df[target_col].isna().sum())
test_target_missing_before = int(test_df[target_col].isna().sum())

missing_before_df = pd.DataFrame(
    {
        "train_missing": train_df.isna().sum(),
        "test_missing": test_df.isna().sum(),
    }
)
print("Missing values before processing:")
print(missing_before_df[missing_before_df.sum(axis=1) > 0] if (missing_before_df

feature_fill_values = train_df[feature_cols].median(numeric_only=True)
train_df[feature_cols] = train_df[feature_cols].fillna(feature_fill_values)
test_df[feature_cols] = test_df[feature_cols].fillna(feature_fill_values)

if train_target_missing_before > 0:
    train_df = train_df.dropna(subset=[target_col]).reset_index(drop=True)
if test_target_missing_before > 0:
    test_df = test_df.dropna(subset=[target_col]).reset_index(drop=True)

missing_after_df = pd.DataFrame(
    {
        "train_missing_after": train_df.isna().sum(),
        "test_missing_after": test_df.isna().sum(),
    }
)
print("\nMissing values after processing:")
print(missing_after_df[missing_after_df.sum(axis=1) > 0] if (missing_after_df.su

X_train = train_df[feature_cols].to_numpy(dtype=np.float32)
y_train = train_df[target_col].to_numpy(dtype=np.float32).reshape(-1, 1)
X_test = test_df[feature_cols].to_numpy(dtype=np.float32)
y_test = test_df[target_col].to_numpy(dtype=np.float32).reshape(-1, 1)

feature_scaler = StandardScaler()
X_train_scaled = feature_scaler.fit_transform(X_train)
X_test_scaled = feature_scaler.transform(X_test)

y_train_log = np.log1p(y_train)
target_scaler = StandardScaler()
y_train_scaled = target_scaler.fit_transform(y_train_log)

val_count = int(len(X_train_scaled) * VAL_RATIO)
train_features = X_train_scaled[:-val_count]
val_features = X_train_scaled[-val_count:]
```

```
train_targets = y_train_scaled[:-val_count]
val_targets = y_train_scaled[-val_count:]

print(f"Train subset: {train_features.shape}, {train_targets.shape}")
print(f"Validation subset: {val_features.shape}, {val_targets.shape}")
print(f"Test subset: {X_test_scaled.shape}, {y_test.shape}")
```

```
Missing values before processing:
No missing values found.

Missing values after processing:
No missing values remain.
Train subset: (701, 8), (701, 1)
Validation subset: (123, 8), (123, 1)
Test subset: (206, 8), (206, 1)
```

5. 构建并训练 MLP

构造三种设置的 MLP 分别是1. 三层MLP，Relu激活函数 2. 两层MLP，Relu激活函数 3. 三层MLP，GELU激活函数

In [45]:

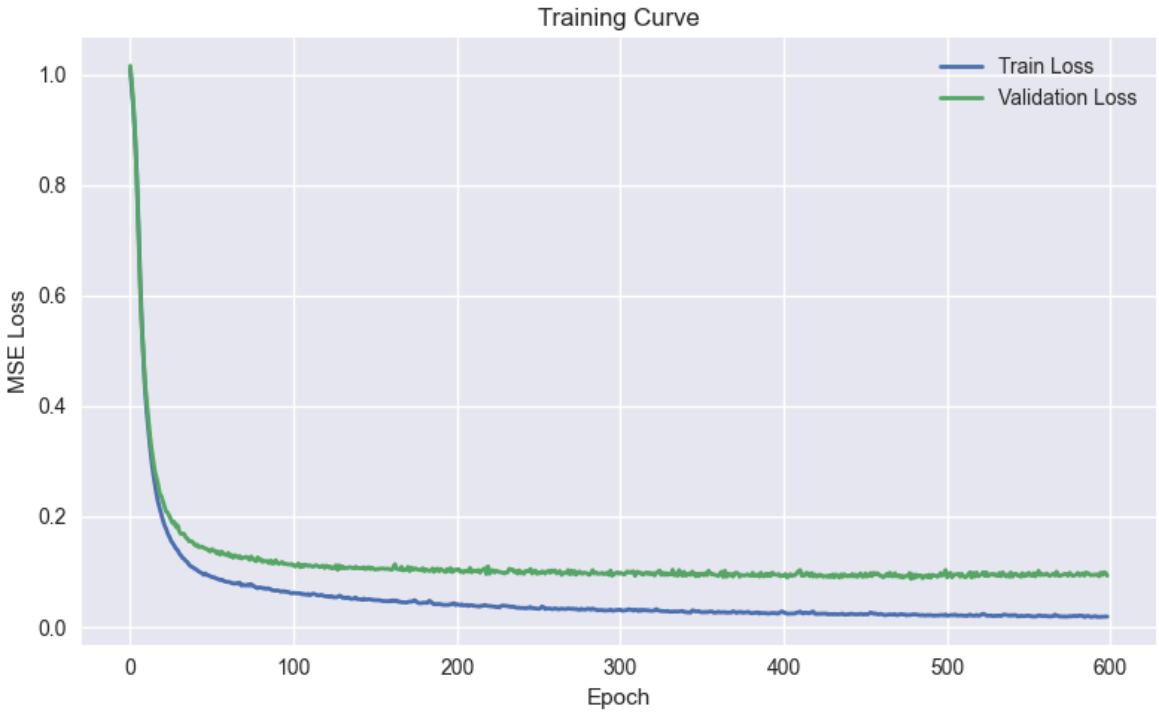
```
train_features_tensor = torch.tensor(train_features, dtype=torch.float32)
val_features_tensor = torch.tensor(val_features, dtype=torch.float32)
train_targets_tensor = torch.tensor(train_targets, dtype=torch.float32)
val_targets_tensor = torch.tensor(val_targets, dtype=torch.float32)
test_features_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)

model = MLPRegressor(input_dim=len(feature_cols))
model1 = MLPRegressor1(input_dim=len(feature_cols))
model2 = MLPRegressor2(input_dim=len(feature_cols))
model, history, best_epoch = train_model(
    model=model,
    train_features=train_features_tensor,
    train_targets=train_targets_tensor,
    val_features=val_features_tensor,
    val_targets=val_targets_tensor,
)
model1, history1, best_epoch1 = train_model(
    model=model1,
    train_features=train_features_tensor,
    train_targets=train_targets_tensor,
    val_features=val_features_tensor,
    val_targets=val_targets_tensor,
)
model2, history2, best_epoch2 = train_model(
    model=model2,
    train_features=train_features_tensor,
    train_targets=train_targets_tensor,
    val_features=val_features_tensor,
    val_targets=val_targets_tensor,
)

print(f"Best epoch: {best_epoch}")
print(f"Best epoch1: {best_epoch1}")
print(f"Best epoch2: {best_epoch2}")
```

```
Best epoch: 479
Best epoch1: 691
Best epoch2: 774
```

6. 训练过程可视化

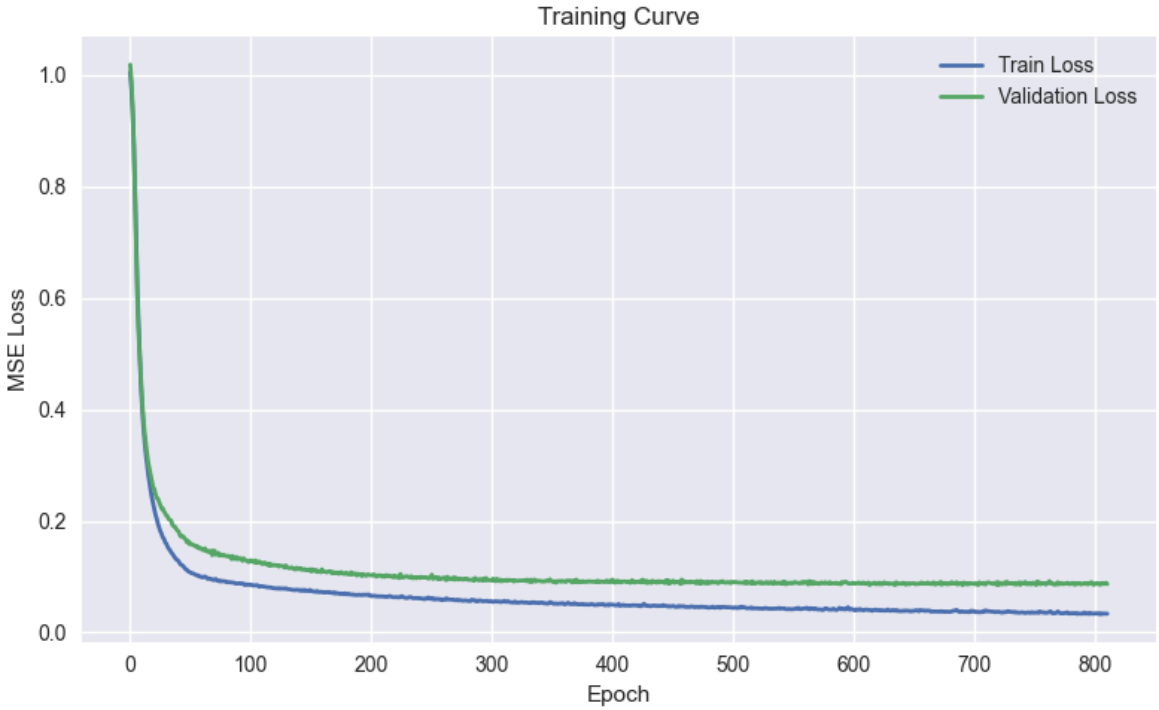
3层MLP，隐藏层 64-32-16-1，激活函数Relu的模型训练情况：

In [46]:
```
plot_loss_curve(history, OUTPUT_DIR)
```



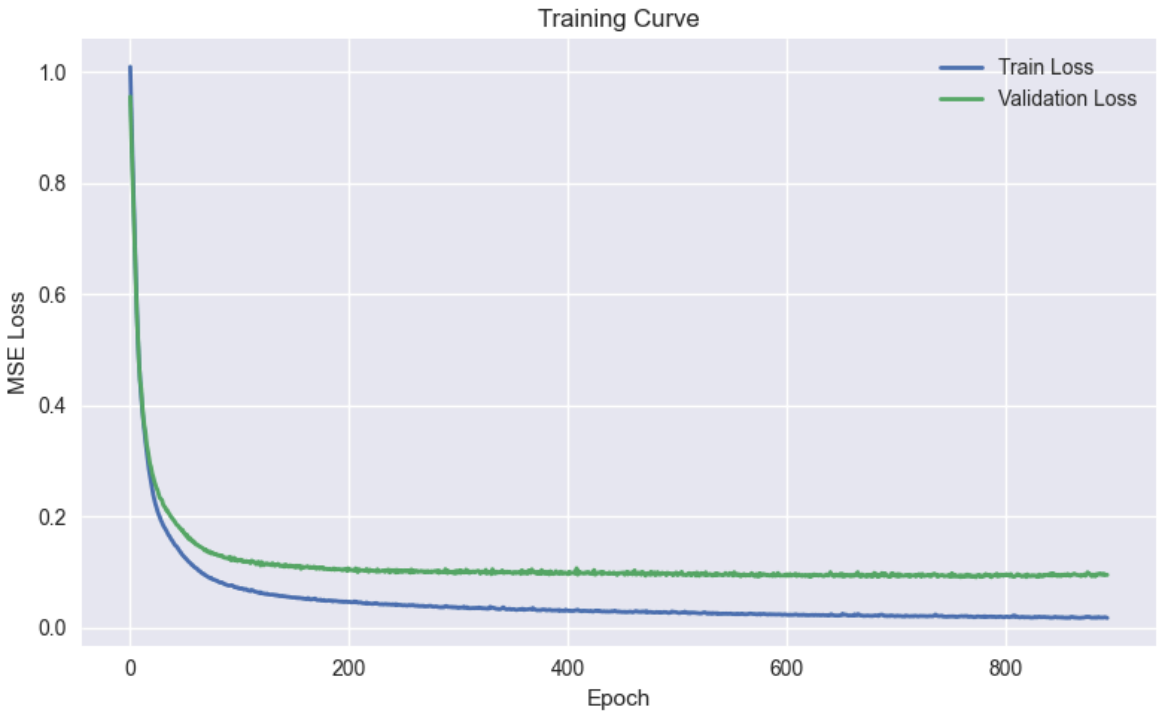
更换激活函数为GELU的模型训练情况

In [47]:
```
plot_loss_curve(history1, OUTPUT_DIR)
```



减少MLP层数为两层模型的训练情况

In [48]:
```
plot_loss_curve(history2, OUTPUT_DIR)
```



7. 测试集预测与评估

In [49]:
```
model.eval()
with torch.no_grad():
    test_predictions_scaled = model(test_features_tensor).cpu().numpy()
    test_predictions_scaled1 = model1(test_features_tensor).cpu().numpy()
    test_predictions_scaled2 = model2(test_features_tensor).cpu().numpy()

test_predictions_log = target_scaler.inverse_transform(test_predictions_scaled)

test_predictions = np.expm1(test_predictions_log)

mse = mean_squared_error(y_test.ravel(), test_predictions.ravel())
mse1 = mean_squared_error(y_test.ravel(), np.expm1(target_scaler.inverse_transfo
mse2 = mean_squared_error(y_test.ravel(), np.expm1(target_scaler.inverse_transfo
r2 = r2_score(y_test.ravel(), test_predictions.ravel())
r2_1 = r2_score(y_test.ravel(), np.expm1(target_scaler.inverse_transform(test_pr
r2_2 = r2_score(y_test.ravel(), np.expm1(target_scaler.inverse_transform(test_pr

baseline = LinearRegression()
baseline.fit(X_train_scaled, y_train.ravel())
baseline_predictions = baseline.predict(X_test_scaled)
baseline_mse = mean_squared_error(y_test.ravel(), baseline_predictions)
baseline_r2 = r2_score(y_test.ravel(), baseline_predictions)

metrics_df = pd.DataFrame(
    {
        "metric": ["MLP 的测试均方误差（MSE）", "MLP 的测试 R2","减少层数 MLP 的
        "value": [mse, r2, mse2, r2_2, mse1, r2_1, baseline_mse, baseline_r2],
    }
```

```
)
metrics_df
```

Out[49]:

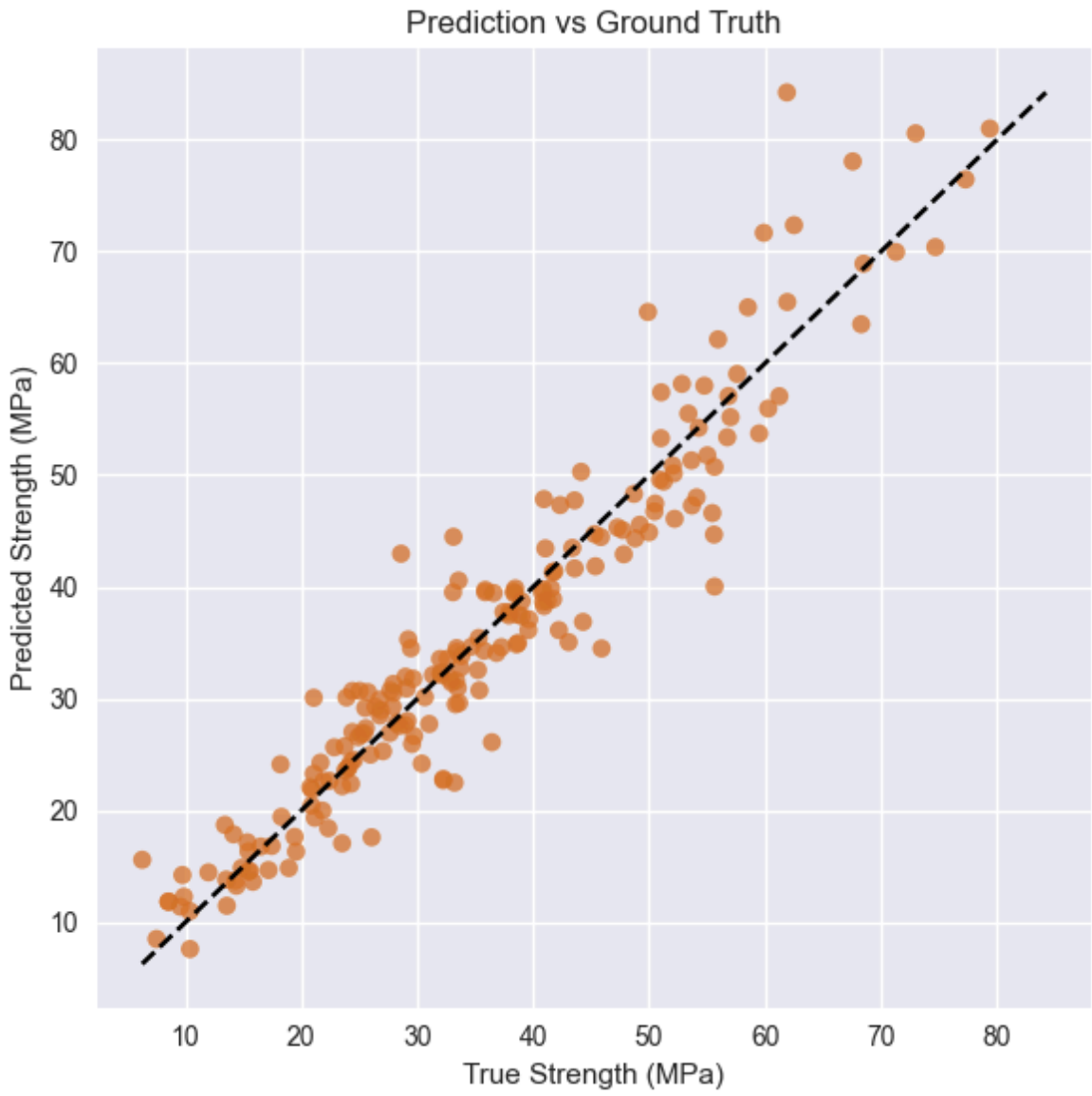
	metric	value
0	MLP 的测试均方误差（MSE）	22.129147
1	MLP 的测试 R2	0.906202
2	减少层数 MLP 的测试均方误差（MSE）	23.329763
3	减少层数 MLP 的测试 R2	0.901113
4	换激活函数 MLP 的测试均方误差（MSE）	22.202463
5	换激活函数 MLP 的测试 R2	0.905891
6	线性回归的测试 MSE	106.880814
7	线性回归的测试 R2	0.546967

8. 预测结果可视化

可视化 MLP 预测结果

In [53]:

```
plot_predictions(y_test.ravel(), test_predictions.ravel(), OUTPUT_DIR)
```



9. 使用相关系数最大的几个特征进行 MLP 预测

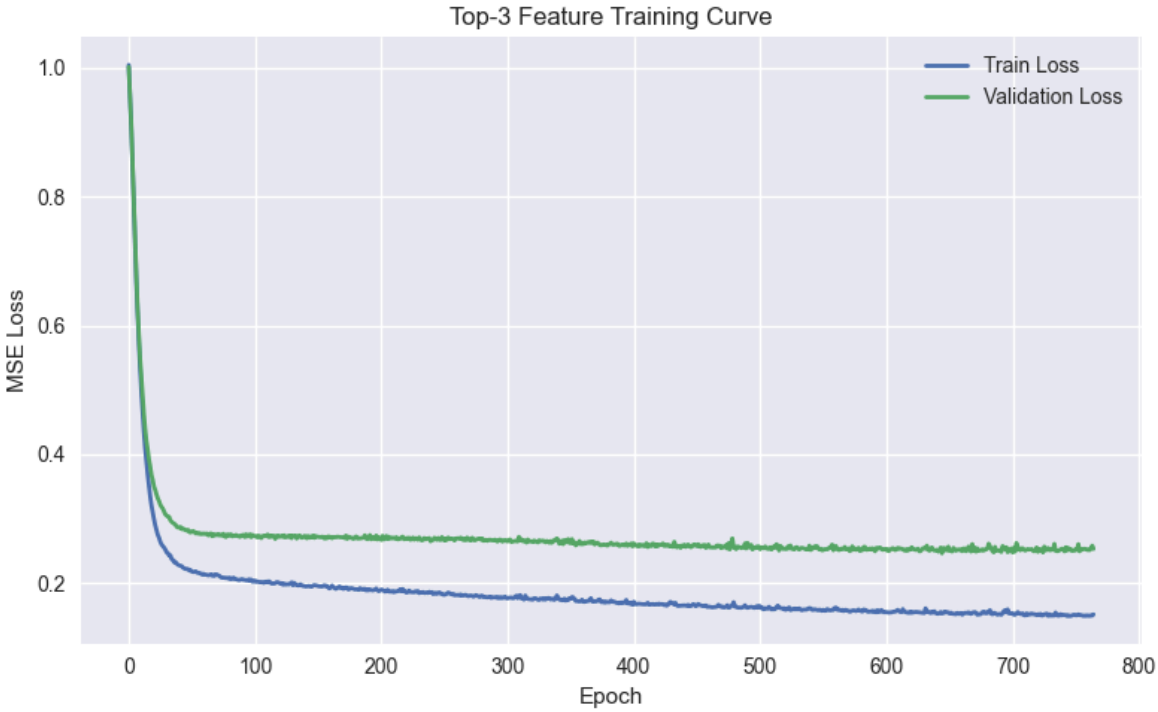
这里选取与目标值相关系数排名前 3 的特征，重复标准化、训练、测试和可视化流程，并与全特征 MLP 结果做对比。

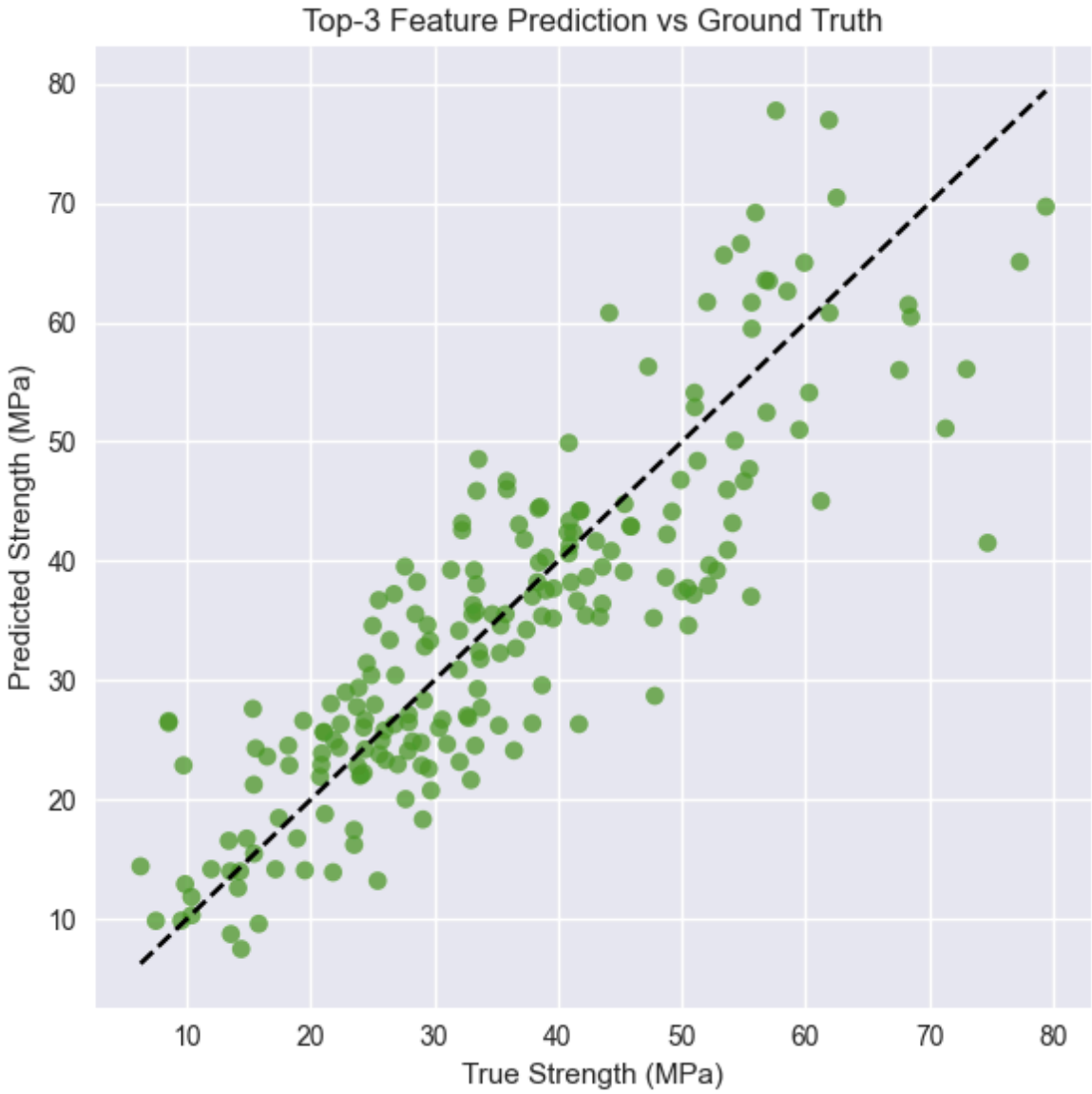
In [54]:
```
top_k = 3
top_features = correlations.head(top_k).index.tolist()
print(f"Top {top_k} correlated features: {top_features}")

X_train_top = train_df[top_features].to_numpy(dtype=np.float32)
X_test_top = test_df[top_features].to_numpy(dtype=np.float32)
y_train_top = train_df[target_col].to_numpy(dtype=np.float32).reshape(-1, 1)
y_test_top = test_df[target_col].to_numpy(dtype=np.float32).reshape(-1, 1)

top_feature_scaler = StandardScaler()
X_train_top_scaled = top_feature_scaler.fit_transform(X_train_top)
X_test_top_scaled = top_feature_scaler.transform(X_test_top)

y_train_top_log = np.log1p(y_train_top)
top_target_scaler = StandardScaler()
y_train_top_scaled = top_target_scaler.fit_transform(y_train_top_log)
```

```
top_val_count = int(len(X_train_top_scaled) * VAL_RATIO)
top_train_features = X_train_top_scaled[:-top_val_count]
top_val_features = X_train_top_scaled[-top_val_count:]
top_train_targets = y_train_top_scaled[:-top_val_count]
top_val_targets = y_train_top_scaled[-top_val_count:]

set_seed(SEED)
top_model = MLPRegressor(input_dim=len(top_features))
top_model, top_history, top_best_epoch = train_model(
    model=top_model,
    train_features=torch.tensor(top_train_features, dtype=torch.float32),
    train_targets=torch.tensor(top_train_targets, dtype=torch.float32),
    val_features=torch.tensor(top_val_features, dtype=torch.float32),
    val_targets=torch.tensor(top_val_targets, dtype=torch.float32),
)

top_model.eval()
with torch.no_grad():
    top_test_predictions_scaled = top_model(torch.tensor(X_test_top_scaled, dtyp

top_test_predictions_log = top_target_scaler.inverse_transform(top_test_predicti
top_test_predictions = np.expm1(top_test_predictions_log)

top_feature_mse = mean_squared_error(y_test_top.ravel(), top_test_predictions.ra
top_feature_r2 = r2_score(y_test_top.ravel(), top_test_predictions.ravel())

comparison_df = pd.DataFrame(
    {
        "model": ["全部特征 MLP", f"Top-{top_k} 相关系数特征 MLP"],
        "测试_mse": [mse, top_feature_mse],
        "测试_r2": [r2, top_feature_r2],
        "最优epoch": [best_epoch, top_best_epoch],
    }
)
comparison_df.to_csv(OUTPUT_DIR / "top_feature_model_metrics.csv", index=False,

top_feature_predictions_df = pd.DataFrame(
    {
        "true_strength_mpa": y_test_top.ravel(),
        "predicted_strength_mpa": top_test_predictions.ravel(),
    }
)
top_feature_predictions_df.to_csv(OUTPUT_DIR / "top_feature_predictions.csv", in

plt.figure(figsize=(8, 5))
plt.plot(top_history["train_loss"], label="Train Loss", linewidth=2)
plt.plot(top_history["val_loss"], label="Validation Loss", linewidth=2)
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title(f"Top-{top_k} Feature Training Curve")
plt.legend()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "top_feature_loss_curve.png", dpi=200)
plt.show()
plt.close()

min_value = min(float(y_test_top.min()), float(top_test_predictions.min()))
max_value = max(float(y_test_top.max()), float(top_test_predictions.max()))

plt.figure(figsize=(6, 6))
```

```
plt.scatter(y_test_top, top_test_predictions, alpha=0.75, color="#4c9a2a", edgec
plt.plot([min_value, max_value], [min_value, max_value], linestyle="--", color="
plt.xlabel("True Strength (MPa)")
plt.ylabel("Predicted Strength (MPa)")
plt.title(f"Top-{top_k} Feature Prediction vs Ground Truth")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "top_feature_prediction_scatter.png", dpi=200)
plt.show()
plt.close()

comparison_df
```

```
Top 3 correlated features: ['Cement (component 1)(kg in a m^3 mixture)', 'Superpl
asticizer (component 5)(kg in a m^3 mixture)', 'Age (day)']
```





Out[54]:

	model	测试_mse	测试_r2	最优epoch
0	全部特征 MLP	22.129147	0.906202	479
1	Top-3 相关系数特征 MLP	64.804893	0.725313	645

10. 总结

- 该 notebook 复现了作业要求中的完整训练、测试和可视化流程
- 数据预处理中新增了缺失值检查、特征中位数填充和目标缺失样本删除
- 除全特征 MLP 外，还增加了基于高相关特征的 MLP 对比实验、多模型参数（隐藏层维度、激活函数类型）实验，结果会额外保存到 `notebook_outputs/`